# pprof в Go

Как найти утечку памяти, пока процесс ещё жив

Инцидент с прода, демо `http_leak`, diff двух профилей

# О чём поговорим

1. Инцидент: память росла, и как её разбирали
2. План разбора: метрики, логи, горутины, pprof
3. Та же утечка в `http_leak` под нагрузкой
4. Diff двух профилей
5. Шпаргалка: heap, CPU, goroutine, diff
6. Частые причины роста памяти

# Память росла до лимита пода

Рестарт возвращал сервис ненадолго.

График памяти – повод открыть разбор. Откуда байты, на нём ещё не написано.



# План разбора

### 1. Метрики: RPS, ошибки, размер тела

Память и горутины — на том же окне. Прямая вверх при ровном RPS и ровном размере тела — накопление. Полка после пика — рабочий объём кэша или пула.

### 2. Логи

Паника, таймауты, обрывы соединений к downstream.

### 3. Состояние горутин

Число и один и тот же стек, повторённый тысячи раз.

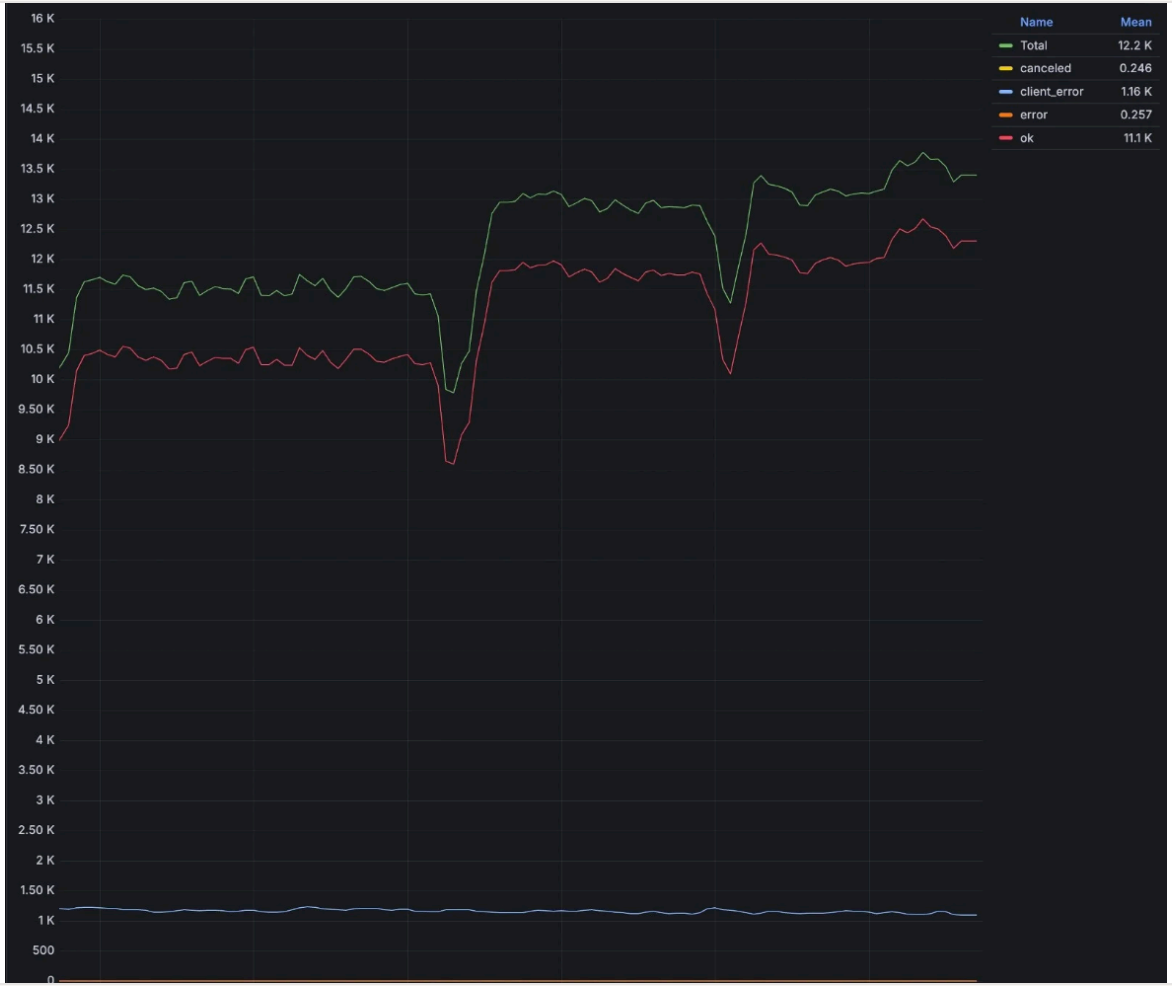
### 4. pprof на проде

heap и goroutine снимаем, пока процесс жив. Дальше Graph и Source.

# Шаг 1. RPS, ошибки, размер тела

RPS объясняет рост только вместе с размером ответа и ошибками.

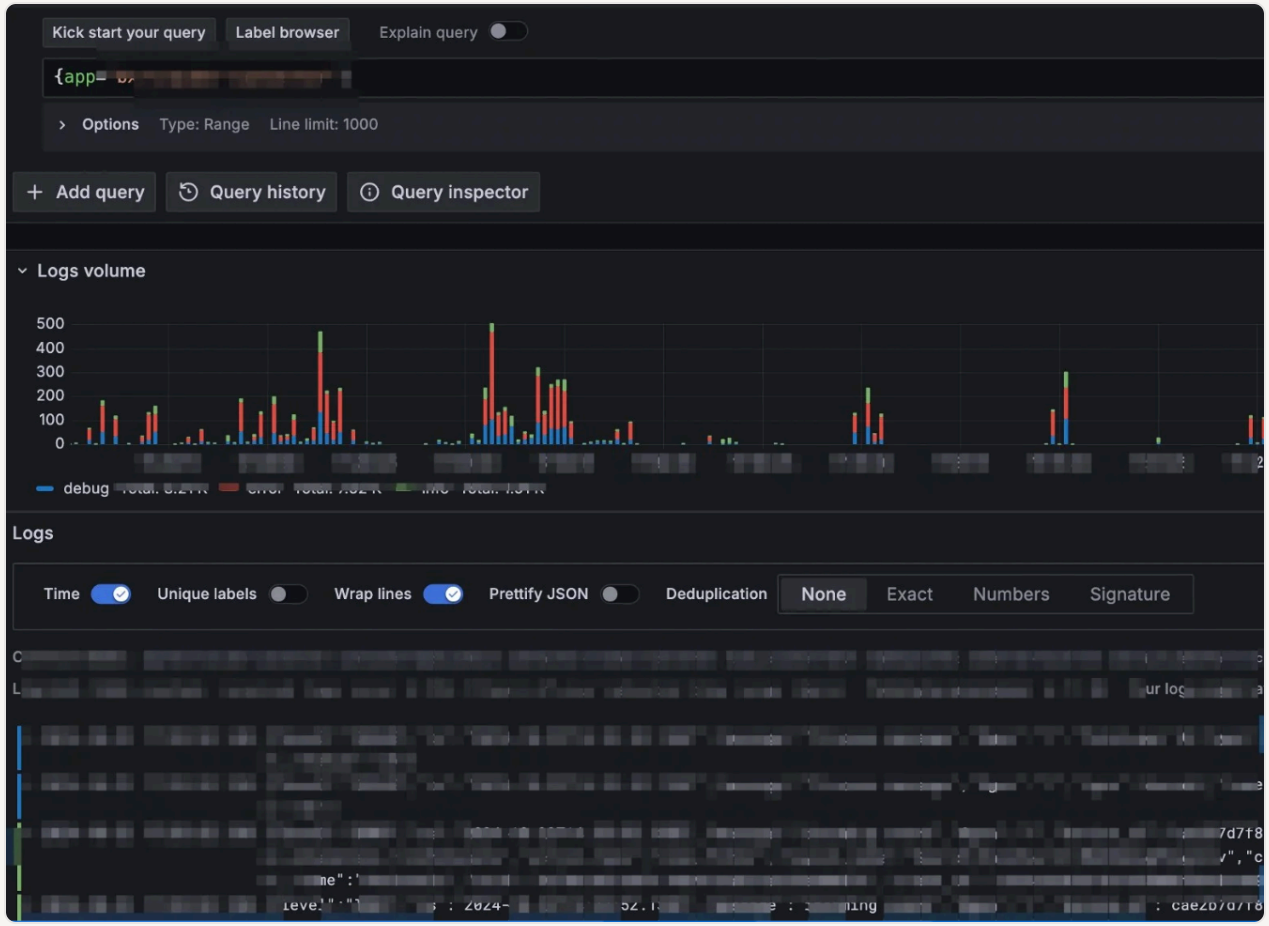
Ровный трафик и растущая память — идём в логи и рантайм, а не в масштабирование.



# Шаг 2. Логи

Ищем панику, таймауты, `connection reset`, рост времени downstream.

Дальше смотрим горутины и в том случае, когда в логах тихо: незакрытый Body часто молчит до исчерпания памяти или файловых дескрипторов.



# Шаг 3. Состояние горутин

- Метрика числа горутин рядом с графиком памяти
- Дамп: `/debug/pprof/goroutine?debug=1`
- Утечка выглядит как один стек с большим счётчиком
- Для незакрытого Body это `net/http.(*persistConn).readLoop`

`debug=2` печатает аргументы кадров. Там бывают URL и заголовки: в общий канал такой дамп лучше не класть.

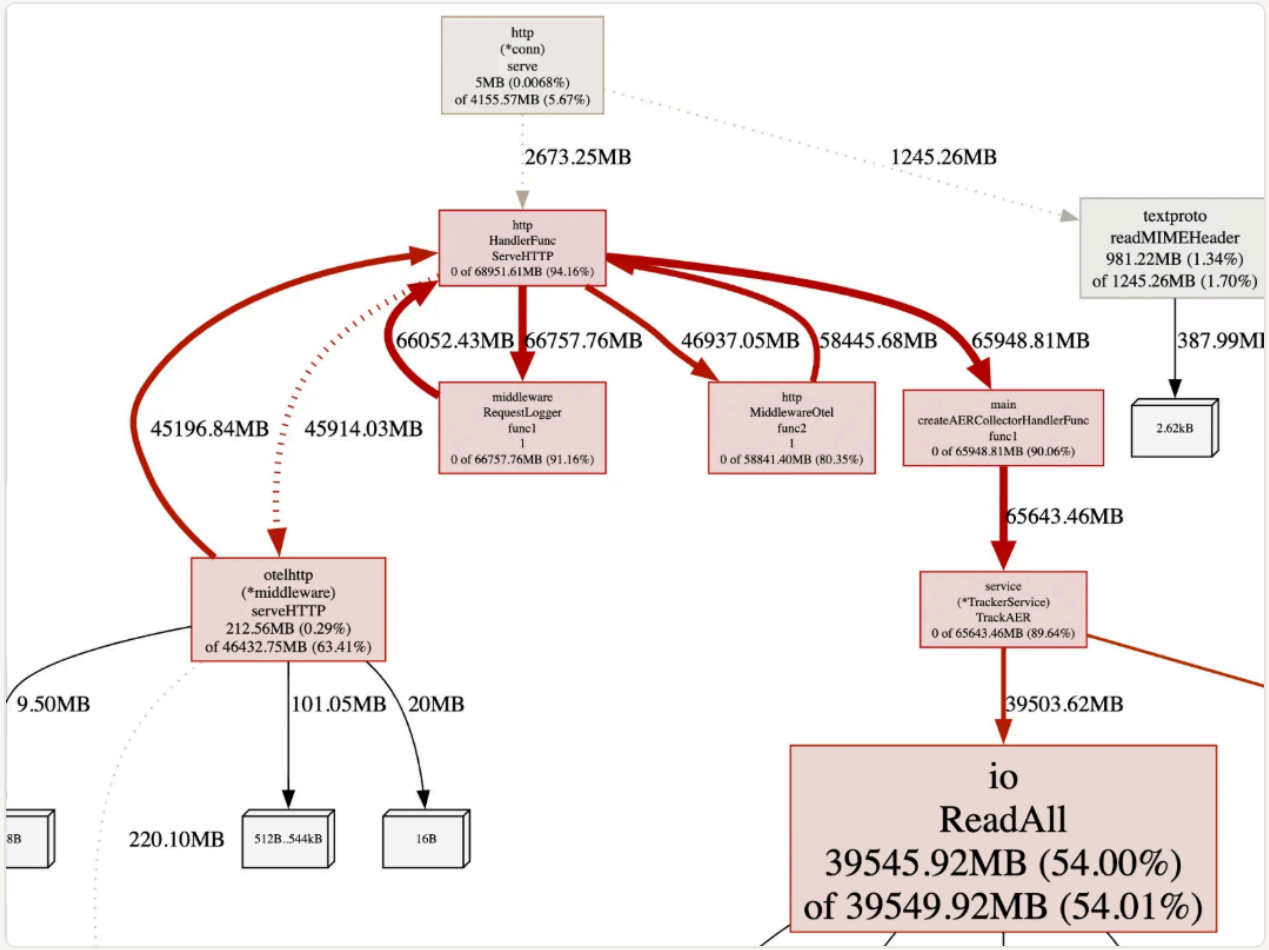
Несколько `readLoop` — обычный пул keep-alive. Тысячи при ровном RPS — соединения не возвращаются.

# Шаг 4. Граф pprof

Снимок heap, режим Graph.

Крупные узлы — функции, которые держат живые объекты. От узла переходим в Source.

Для Graph на машине нужен `dot` из Graphviz. Top и Source работают и без него.



# Шаг 4. Режим Source

Graph называет функцию. Source показывает строку аллокации.

Строка в профиле — место, где память выделили. Вызов, который забыл `Close`, может быть уже выше по стеку или уже вернулся.

**io.ReadAll**

/usr/local/go/src/io/io.go

```
Total:     874.21MB   874.21MB (flat, cum) 91.14%
   690            .          .                   Reader
   691            .          .           }
   692            .          .
   693            .          .           func (nopCloser) Close() error { r
   694            .          .
   695            .          .           type nopCloserWriterTo struct {
   696            .          .                   Reader
   697            .          .           }
   698            .          .
   699     874.21MB   874.21MB           func (nopCloserWriterTo) Close() e
   700            .          .
   701            .          .           func (c nopCloserWriterTo) WriteTo
   702            .          .                   return c.Reader.(WriterTo).W
   703            .          .           }
   704            .          .
   705            .          .           // ReadAll reads from r until an e
   706            .          .           // A successful call returns err =
```

## Причина: Body ответа не закрыт

Исходящий ответ прочитали и не закрыли. Соединение осталось за процессом вместе с горутинами `net/http`.

# Почему это держит память

`Client.Get` поднимает постоянное соединение. `readLoop` и `writeLoop` ждут, пока `resp.Body` закроют.

GC их не забирает: стек горутины — живой корень. Пока клиент не читает тело, downstream тоже стоит в `conn.serve`.

На один такой запрос в процессе остаются:

- две горутины клиента и горутина сервера
- буферы `bufio`
- пара файловых дескрипторов

`Get` идёт с `context.Background`. Отмена входящего запроса это соединение не закрывает.

# Демо: `http_leak`

Та же ошибка. Нагрузка снаружи, чтобы на счётчиках и в pprof было видно, как течь растёт.

# Как устроено демо

```
k6 --> /leak или /fixed --> http_leak --> downstream
                                 |
                                 + pprof  127.0.0.1:8080/debug/pprof/
                                 + лог    goroutines, heap_inuse
```

- k6 закрывает ответ `http_leak` и сам память не копит
- `/leak` оставляет Body downstream открытым
- `/fixed` дочитывает тело до EOF и закрывает его
- downstream живёт в том же процессе, поэтому его стек тоже виден в pprof

# Нагрузка

```
TARGET_PATH=/fixed DURATION=30s docker compose up -d
docker compose down && docker compose up -d
```

Второй запуск бьёт в `/leak` две минуты с 50 RPS. Лог: `docker compose logs -f http_leak`.

```
goroutines= 1600 (+1597)  heap_inuse=  18MiB  stack= 9216KiB
```

`(+N)` считается от старта процесса. `GET /gc` делает сборку мусора и печатает те же поля.

`/fixed` и `/leak` сравниваем на свежем процессе: уже открытые соединения живут до его завершения.

# Что видно в логе при 50 RPS

`/fixed` держится на полке:

```
goroutines=   10 (   +7)  heap_inuse=   1MiB
```

`/leak` прибавляет около 300 горутин на строку. 50 запросов в секунду × 3 горутины × 2 секунды:

```
goroutines=    5 (   +2)  heap_inuse=   1MiB
goroutines=  400 ( +397)  heap_inuse=   6MiB
goroutines= 1600 (+1597)  heap_inuse=  18MiB
goroutines= 3305 (+3302)  heap_inuse=  28MiB
```

Последняя строка снята после остановки нагрузки и GC. Счётчик остался.

# Дырка и рабочий вызов

```go
func leakyCall(url string) error {
    resp, err := client.Get(url)
    if err != nil {
        return err
    }
    _ = resp // Body не закрываем
    return nil
}

func fixedCall(url string) error {
    resp, err := client.Get(url)
    if err != nil {
        return err
    }
    defer resp.Body.Close()
    _, err = io.Copy(io.Discard, resp.Body)
    return err
}
```

`Close` отпускает соединение. Чтение до EOF возвращает его в пул.

# Какой профиль открывать

| Вопрос | Профиль |
|---|---|
| Кто не завершился | `goroutine` |
| Какие объекты ещё живы | `heap`, sample `inuse_space` |
| Кто давит на GC | `heap`, sample `alloc_space`, или `/allocs` |
| Кто ест CPU | `/debug/pprof/profile?seconds=20` |

```
go tool pprof -http=127.0.0.1:9091 http://127.0.0.1:8080/debug/pprof/goroutine
go tool pprof -http=127.0.0.1:9090 http://127.0.0.1:8080/debug/pprof/heap
```

На этой утечке CPU-профиль почти пустой: `readLoop` спит на сокете. Её показывает профиль горутин.

# Diff двух профилей

Пока k6 стучит в `/leak`:

```
curl -fsS http://127.0.0.1:8080/debug/pprof/goroutine \
  -o profiles/goroutine1.pb.gz
sleep 20
curl -fsS http://127.0.0.1:8080/debug/pprof/goroutine \
  -o profiles/goroutine2.pb.gz
```

```
go tool pprof -top -cum -base \
  profiles/goroutine1.pb.gz profiles/goroutine2.pb.gz

go tool pprof -http=127.0.0.1:9091 -diff_base \
  profiles/goroutine1.pb.gz profiles/goroutine2.pb.gz
```

`-base` вычитает первый файл из второго: в `-top` колонка cum – сколько стеков прибавилось.

`-diff_base` нужен веб-интерфейсу. В текстовом top проценты там легко больше 100%: знаменатель – размер базового снимка. Читайте абсолютные числа.

# Как читать этот diff

Три стека, которые расходятся при 50 RPS примерно на число запросов за окно:

- `net/http.(*persistConn).readLoop` – клиент ждёт чтения Body
- `net/http.(*persistConn).writeLoop` – клиент держит соединение
- `net/http.(*conn).serve` – downstream в том же процессе

За 20 секунд это около тысячи каждого.

В heap по `inuse_space` рядом растут `bufio.NewReaderSize`, `bufio.NewWriterSize` и `Transport.dialConn`.

`alloc_space` за ту же паузу часто показывает сам сбор профиля: `runtime/pprof`, `compress/flate`. Для утечки берём `inuse_space`.

В демо куча сэмплируется каждые 64 КБ. На проде значение по умолчанию – 512 КБ, мелкие буферы попадают реже. Профиль горутин учитывает все горутины.

# Шпаргалка pprof

| Профиль | Откуда | Зачем |
|---|---|---|
| **heap** | `/debug/pprof/heap` | Живые объекты. Утечка – рост `inuse_space` |
| **cpu** | `/debug/pprof/profile?seconds=20` | Кто занимает процессор |
| **goroutine** | `/debug/pprof/goroutine` | Какие стеки не завершаются |
| **diff** | `-base` в терминале, `-diff_base` в UI | Что изменилось между двумя снимками |

Режимы UI: **Top**, **Graph**, **Flame Graph**, **Source**.

`allocs` / `alloc_space` включают уже собранный мусор: так смотрят давление на GC.
CPU-профиль на 20 секунд заметно грузит процесс. Снимайте pprof с localhost или с admin-порта.
Ещё есть `block` и `mutex`: оба выключены, пока их явно не включат в рантайме.

# Откуда берётся memory leak

1. **Горутины** запускаются на запрос и не выходят: канал, `WaitGroup`, вызов без таймаута
2. **`http.Client` / `Transport`** создаётся на каждый запрос
3. **`resp.Body`** не закрыт
4. **`sql.Rows` и файлы** не закрыты, соединение или дескриптор остаётся занятым
5. **Map, кэш, очередь** без верхней границы и без TTL
6. **`time.Tick`** или `NewTicker` без `Stop` в функции, которую зовут много раз
7. **Короткий срез** удерживает большой backing array
8. **`cancel` не вызван.** `WithCancel` от долгого родителя держит дочерний контекст, пока жив родитель. Горутина на `ctx.Done()` не выходит. `WithTimeout` без `cancel` держит таймер до дедлайна

Прямая вверх при ровном RPS – утечка. Полка после пика чаще всего пул, кэш или `sync.Pool`.

# Как это выглядит в коде

## Горутина переживает запрос

```
go func() { out <- fetch(context.Background()) }()
```

Канал никто не читает. Всё, что замыкание захватило, живёт вместе со стеком.

## Свой Transport на запрос

```
c := &http.Client{Transport: &http.Transport{}}
```

У нулевого `Transport` `IdleConnTimeout` равен 0: простаивающие соединения не закрываются. Клиент создают один раз при старте.

## `cancel` не вызван

```
ctx, cancel := context.WithCancel(appCtx)
go func() { <-ctx.Done() }()
```

Пока жив `appCtx`, живы и контекст, и горутина. `cancel()` отпускает оба.

# Что уносим с собой

1. Метрики: память против RPS, ошибок и размера тела
2. Логи, затем число и стеки горутин
3. pprof с живого процесса: goroutine, heap, Source
4. Diff двух снимков при том же RPS
5. На ревью: один `Transport`, `defer resp.Body.Close()`, чтение до EOF

Если процесс уже перезапустился, разовый профиль пустой. Постоянный сбор тех же профилей оставляет снимки до рестарта.